



Data Observability & Lineage

*AICB-P2T2 · Ngày 27 · Chương 6: Tổng Hợp
· Quan sát chính DỮ LIỆU, không chỉ hệ thống*

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 6

HÃY SUY NGHĨ...

*“Pipeline báo **thành công**. Dashboard vẫn xanh.
Nhưng **74%** đội ngũ data nói: người đầu tiên
phát hiện dữ liệu sai là **business stakeholder**
— không phải hệ thống giám sát của họ.*

*Dashboard xanh không có nghĩa là dữ liệu
đúng. Hôm nay: quan sát chính **DỮ LIỆU**.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Tại sao quan sát DATA, không chỉ hệ thống
2. 7 chiều của data observability
3. Bức tranh thị trường 2026
4. Bộ công cụ OSS chạy ngay (GX 1.0, Soda 4.0)
5. Nền tảng SaaS & trực rules-vs-ML
6. Drift & anomaly detection (PSI/KS/KL/MMD)
7. Data Contracts I — chuẩn ODCS
8. Data Contracts II — thực thi & shift-left
9. Lineage I — OpenLineage & mô hình sự kiện
10. Lineage II — catalog, column-level, impact
11. AI-Infra I — Feature Stores (skew)
12. AI-Infra II — Vector/Embedding & RAG
13. AI-Infra III — Training/LLM/Agent data
14. Frontier & chi phí (lakehouse, streaming)
15. Tuân thủ (PDPL), sự cố & Lab 27

Ranh giới với Ngày 23

Ngày 23 quan sát **HỆ THỐNG** (service healthy?). Ngày 27 quan sát **DỮ LIỆU** (data đúng/tươi/đầy đủ/truy vết được?). Không dạy lại Prometheus/SLO burn-rate.

- **Hiểu (Understand):** phân biệt data observability vs pipeline/system monitoring; 7 chiều chất lượng dữ liệu.
- **Áp dụng (Apply):** viết checks bằng Great Expectations 1.0 + Soda 4.0; tính PSI/KS/MMD để phát hiện drift.
- **Phân tích (Analyze):** đọc lineage để làm *impact analysis* (blast radius) & *root-cause*.
- **Thiết kế (Create):** một *data contract* (ODCS) + quan sát skew của feature store & embedding drift của RAG.
- **Đánh giá (Evaluate):** chọn công cụ (build-vs-buy, rules-vs-ML) cho từng tình huống.

Tư duy cốt lõi

Schema checks chỉ bắt được 7,8% sự cố dữ liệu thật. Phần còn lại cần freshness, volume, distribution, lineage, contracts & ML anomaly detection.

Artifact cần nộp (Lab 27)

Một bộ data-observability chạy được zero-key trên DuckDB + lineage graph trên Marquez.

- GX 1.0 / Soda 4.0 checkpoint chặn pipeline khi data vi phạm (freshness/volume/distribution/schema).
- Một **data contract** `.odcs.yaml` + `datacontract test pass/fail` trong CI.
- Phát sinh **OpenLineage** events → Marquez UI; trả lời “cột này hỏng thì ai chịu ảnh hưởng?”.
- Một bài kiểm tra **training-serving skew** (PSI) trên feature store + **embedding drift** (domain-classifier AUC) cho corpus RAG.

Tại sao quan sát DỮ LIỆU, không chỉ hệ thống

Pipeline “thành công” không có nghĩa là data đúng. Schema checks bắt < 8% sự cố.

01

Pipeline & System Monitoring (Ngày 23)

Góc nhìn hạ tầng — “service có khỏe không?”

- Job có chạy? Mất bao lâu? Có lỗi exit?
- CPU/RAM/GPU, p99 latency, error rate
- Prometheus, Grafana, OTel, SLO burn-rate

→ Tất cả **xanh** mà data vẫn có thể **sai**.

Data Observability (Ngày 27)

Góc nhìn dữ liệu — “data có ĐÚNG không?”

- **Fresh?** (tươi) **Complete?** (đủ records)
- **Distribution?** (phân phối hợp lý) **Schema?**
- **Lineage?** (truy vết) **Contract?** (đúng cam kết)

→ Bắt **silent data issues** TRƯỚC khi model / agent / dashboard tiêu thụ.

Phân tích nguyên nhân gốc 2026 của Monte Carlo trên **11 triệu+ bảng**: schema drift — thứ hầu hết mọi người dựng đầu tiên — chỉ gây **7,8%** sự cố.

Nguyên nhân gốc của sự cố dữ liệu	%	
Lỗi pipeline (logic biến đổi)	26,2%	
Biến động dữ liệu thực tế (distribution)	20,0%	
Ingestion (nguồn vào)	16,6%	
Platform/hạ tầng	15,2%	
Thay đổi chủ ý / backfill	14,2%	
Schema drift	7,8%	

Nguồn: Monte Carlo, 2026 Data Quality Statistics (11M+ bảng) — trích dẫn là số liệu của Monte Carlo.

Công thức (analog của MTTD/MTTR)

$$\text{Data Downtime} = N_{\text{incidents}} \times (\text{TTD} + \text{TTR})$$

TTD = time-to-detect, TTR = time-to-resolve. Mục tiêu: giảm cả ba thừa số.

- **74%** đội ngũ: business phát hiện sự cố *trước* (so với 47% năm 2022).
- **68%** sự cố mất ≥ 4 giờ chỉ để *phát hiện*.
- TTR trung bình $\approx 9\text{--}15$ giờ tùy nguồn (9h = số thận trọng, Monte Carlo 2022; chi tiết ở §10); ≈ 1 sự cố / 10 bảng / năm.

Lưu ý: “Tất cả dashboard xanh” + “khách hàng gọi báo số liệu sai” = bạn đang thiếu **data observability**, không phải thiếu system monitoring.

Số liệu thường gặp	Nguồn thật	Cảnh báo
\$12,9M/năm chi phí data quality kém	Gartner (Melody Chien)	số 2021 , mẫu lệch enterprise
\$3,1 ngàn tỷ/năm (US)	Redman, HBR 2016 (không phải IBM)	1 thập kỷ, phương pháp mờ
Data engineer tốn ~40% thời gian vì data xấu	Monte Carlo/Wakefield 2022 (n=300)	dùng được
“80% thời gian để dọn data”	CrowdFlower/Forbes 2016	folklore — dùng 40%

Bài học nghề

Luôn **truy về nguồn gốc** của một con số trước khi đưa lên slide — nhiều “stat kinh điển” là tam sao thất bản.

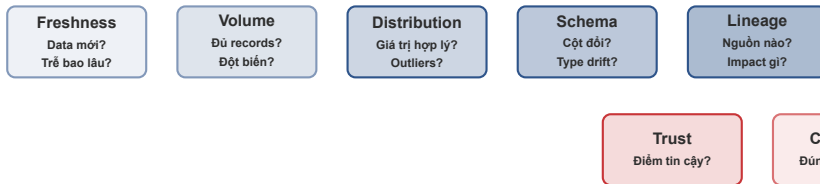
Sự cố	Nguyên nhân dữ liệu	Hậu quả
Unity Software (5/2022)	Ingest training data xấu từ 1 khách lớn	−\$110M guidance, cổ phiếu −37% (~\$5B)
Equifax (2022)	Lỗi code → điểm tín dụng sai ~2,5M người	NY AG phạt \$725K (1/2025) + giám sát
Zillow Offers (11/2021)	Model/data drift vs thị trường biến động	writedown >\$500M, sa thải ~25%
Citigroup (4/2024)	Nhập liệu: số 0 không bị xoá → \$81 ngàn tỷ	“near-miss”, qua 2 checker, bắt sau ~90 phút

Mẫu số chung

Cả 4 đều *không* phải lỗi “service down”. Pipeline chạy ngon; **nội dung dữ liệu** mới là thứ sai. Citigroup ⇒ ưu tiên **automated range/anomaly checks** hơn review thủ công.

7 chiều của Data Observability

Từ 5 trụ cột của Monte Carlo, mở rộng sang contracts & trust.



5 trụ cột gốc (Barr

Moses/Monte Carlo, 2019): Freshness · Volume · Distribution · Schema · Lineage. **+2 chiều 2026:** *Contract-conformant* (đúng hợp đồng dữ liệu) & *Trustworthy* (điểm tin cậy tổng hợp).

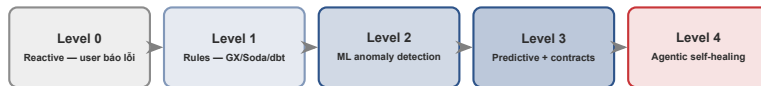
6 chiều DQ cổ điển (DAMA)

- **Accuracy** — đúng giá trị thực
- **Completeness** — không thiếu (null rate)
- **Consistency** — khớp giữa các nguồn
- **Timeliness** — kịp thời (= freshness)
- **Validity** — đúng định dạng/miền giá trị
- **Uniqueness** — không trùng lặp

Ánh xạ sang 5 trụ cột observability

- Timeliness → **Freshness**
- Completeness/Uniqueness → **Volume**
- Accuracy/Validity → **Distribution**
- (cấu trúc) → **Schema**
- (xuất xứ) → **Lineage**

DQ = “data có tốt không” (trạng thái). Observability = “làm sao BIẾT khi nó hết tốt” (đo liên tục).



Hầu hết đội ngũ ở **Level 0–1**. Mục tiêu

khoá này: vững Level 1 (rules + contracts), đạt **Level 2** (ML anomaly), hiểu lộ trình Level 3–4 (agentic DQ — xem §14).

Bức tranh thị trường 2026

Hợp nhất mạnh, mọi tool gắn AI agent, contracts “ăn” checks.

03

Nhóm	Thành viên & ghi chú
SaaS độc lập, đang sống	Monte Carlo, Anomalo, Bigeye, Sifflet, Soda Cloud — đều “data + AI”
Bị mua	Metaplane → Datadog (~4/2025) — gộp vào hãng system-observability
Xoay trục	Datafold — <i>sunset</i> OSS data-diff (5/2024), chuyển sang “AI for data teams”
OSS-first / dbt-native	Great Expectations (GX Core), Soda Core, Pandera, Elementary, dbt-expectations

Quy mô thị trường (dùng KHOẢNG, không dùng điểm)

Data observability $\approx$ \$3,1B (2025–26) $\rightarrow$ \$6–8B (2031–34), CAGR $\sim$ 11–12% (Mordor / Fortune BI). **Đừng nhầm** với thị trường “observability” hệ thống/APM ($\sim$ \$28–34B).

Hội tụ ở tầng vendor

- **Metaplane** → **Datadog**: hãng APM mua tool data observability.
- Mọi vendor SaaS lớn đang gắn thêm “agent” AI (dự đoán / ghi nhớ / tự sửa) lên nền tảng sẵn có — chi tiết kỹ thuật (Monte Carlo Agent Observability, Sifflet Sentinel/Sage/Forge) → xem §5.

Lưu ý: Nhưng **công việc** vẫn khác nhau: Metaplane (dù thuộc Datadog) vẫn quan sát **bảng/schema/phân phối** — không phải CPU/latency của service. Đừng để marketing “unified” xoá mất ranh giới data-vs-service.

*“First/only to unify data+AI” là **định vị marketing**, không phải sự thật đã kiểm toán — trình bày như vậy trên lớp.*

Bộ công cụ OSS chạy ngay — \$0

Great Expectations 1.0, Soda 4.0, Pandera, Elementary, dbt tests.

04

```
import great_expectations as gx
context = gx.get_context()           # ephemeral|file|cloud
ds    = context.data_sources.add_pandas("src")
asset = ds.add_dataframe_asset("orders")
bd    = asset.add_batch_definition_whole_dataframe("bd")
suite = context.suites.add(gx.ExpectationSuite("orders"))
suite.add_expectation(gx.expectations
    .ExpectColumnValuesToNotBeNull(column="user_id"))
suite.add_expectation(gx.expectations
    .ExpectColumnValuesToBeBetween(column="amount",
                                   min_value=0, max_value=1e6))
vd = context.validation_definitions.add(gx.ValidationDefinition(
    data=bd, suite=suite, name="vd"))
cp = context.checkpoints.add(gx.Checkpoint(
    name="cp", validation_definitions=[vd], actions=[]))
print(cp.run(batch_parameters={"dataframe": df}).success)
```

Chuỗi chuẩn 1.0

get_context → source/asset → **batch definition** → suite → **validation definition**
→ checkpoint.run().

Lưu ý: GX 1.0 (28/8/2024)
phá vỡ tương thích. ~90%
tutorial online là tiền-1.0
(YAML config, Validator,
batch_request) — **sẽ KHÔNG**
chạy. Phiên bản hiện tại
~1.18.x.

Thay đổi lớn (4.0.5, 28/1/2026)

Trích release notes: “*introduces Data Contracts as the **default** way to define data quality rules*” — **breaking change**, chuyển từ “checks language” sang cú pháp dựa-contract. Có tool tự chuyển đổi từ SodaCL v3.

Bổ sung AI & vận hành

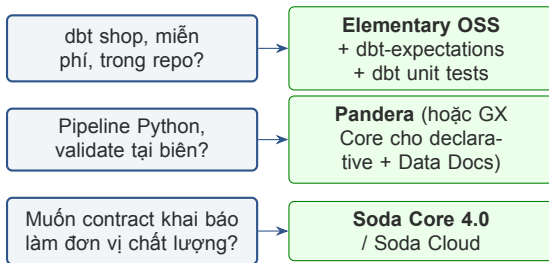
- **Contract Copilot** — viết rule bằng ngôn ngữ tự nhiên
- **Contract Autopilot** — tự sinh (Private Preview)
- **Smart Anomaly Treatment**
- Soda Agent đổi tên → **Soda Runner** (chạy in-VPC)

“70% ít false-positive hơn Prophet” là **benchmark của Soda**, chưa được kiểm chứng độc lập — ghi rõ trên slide.

Công cụ	Là gì	Dùng khi
Pandera	Validate DataFrame trong code (DataFrameModel)	boundary check trong pipeline Python; hỗ trợ Polars /PySpark
Elementary	Gói dbt + CLI ghi kết quả test vào warehouse	dbt shop muốn report + Slack alert miễn phí
dbt-expectations	50+ assertion kiểu GX cho dbt	(do Metaplane duy trì → một mắt xích hợp nhất)
dbt unit tests	GA dbt 1.8 (5/2024): test <i>LOGIC</i> với fixture tĩnh	kiểm tra biến đổi <i>TRU'ỚC</i> khi materialize

Phân biệt sống còn

Unit test = kiểm tra **LOGIC biến đổi** (cho input X có ra output Y?). **Data test/observability** = quan sát **TRẠNG THÁI dữ liệu thật** (data đã materialize có đúng?). Cả hai đều cần; không cái nào thay cái nào.



Nguyên tắc

Rules-based (GX/Soda/dbt) = bạn *BIẾT* “tốt” trông như thế nào. **ML-based** (§5) = để máy học “bình thường” rồi báo lệch.

Nền tảng SaaS & trực Rules-vs-ML

ML tự học ngược; agent tự dự đoán/sửa. Khi nào nên mua?

Nền tảng	Điểm nhấn kỹ thuật	Định vị
Monte Carlo	ML anomaly + lineage; tác giả “5 trụ cột”	category-definer; Agent Observability
Anomalo	Unsupervised, tìm “unknown unknowns” mức record	DQ cho GenAI/unstructured (RAG corpora)
Bigeye	Autometrics + Autothresholds (ML đặt ngưỡng)	“AI Trust Platform”
Sifflet	3 agent: Sentinel/Sage/Forge	agentic; giá theo asset

Khái niệm cốt lõi cần nhớ

Autothresholds: ML học khoảng cảnh báo từ lịch sử → bạn không phải viết ngưỡng tĩnh. **Agentic**: predict (Sentinel) → remember (Sage) → fix (Forge).

Đã có câu trả lời OSS ở §4 (dbt shop → Elementary; validate biên Python → Pandera/GX Core; contract khai báo → Soda Core). Bảng dưới chỉ thêm các tình huống cần trả tiền:

Tình huống	Lựa chọn
ML auto-detect nhiều bảng + lineage + UI sự cố, có ngân sách	Monte Carlo / Bigeye / Sifflet / Anomalo
Đã dùng Datadog	Metaplane-by-Datadog
Corpora GenAI/unstructured	Anomalo
Kiểm tra refactor không đổi output (value diff)	dbt unit tests (data-diff OSS đã chết)

Lưu ý: Build-vs-buy: OSS = \$0 nhưng bạn tự viết/chạy checks; SaaS = ML autodetect + lineage + incident UI, tính tiền theo bảng/asset/volume. Đa số OSS đều có anh em “cloud” (open-core).

Drift & Anomaly Detection — lỗi thống kê

Vượt ngưỡng tĩnh: bộ phát hiện phân tầng + chọn đúng phép đo khoảng cách.

Model	Bản chất	Dùng khi
Online Z-score (def 2.5)	lệch chuẩn, đã de-trend	univariate rẻ
Online MAD (2.5)	median abs deviation	univariate, kháng outlier
Rolling Quantile	10–100 điểm tiến hoá	ngưỡng động không giả định phân phối
Holt-Winters	1 xu hướng + 1 mùa vụ	chuỗi có trend+season
Prophet / STL	nhiều chu kỳ mùa vụ, change-points	seasonality phức tạp
Isolation Forest	tuyến tính, đa biến, no-normality	high-cardinality, đa biến

Tham chiếu: VictoriaMetrics anomaly-detection (OSS). Quy tắc: MAD/IQR/z-score cho univariate rẻ; Prophet/Holt-Winters khi có season+trend; Isolation Forest cho đa biến.

Vấn đề

Ngưỡng tỉnh báo động giả mỗi cuối tuần (volume giảm) hay mùa thuế (spike). → Đội ngũ *alert fatigue* → tắt cảnh báo.

Giải pháp

Trừ đi thành phần **trend + seasonal** (Prophet/STL) TRƯỚC, rồi mới phát hiện bất thường trên phần dư.
Databricks DQM (2/2026): “học mẫu lịch sử & hành vi mùa vụ”.

~40%

ít bảo trì hơn: anomaly monitors
1,33 lần chạm/bảng vs 2,03 (validation tests) vs 2,35 (custom SQL)

Số liệu của Monte Carlo (11M+ bảng, 2026).

Tình huống	Phép đo
numeric, N nhỏ (<1000)	KS
numeric, N lớn	PSI / Wasserstein
categorical	Chi-square / JS
embeddings (high-dim)	MMD / domain-classifier

Lưu ý: **KS** cực nhạy khi >1000 dòng — hãy *sample* trước. **PSI** không phụ thuộc cỡ mẫu.

```
# PSI: tabular, sample-size independent
# PSI = sum( (A_i - E_i) * ln(A_i / E_i) )
# <0.1 no shift | 0.1-0.25 moderate | >0.25 significant
# KS: max gap between two empirical CDFs
# D = sup_x | F_cur(x) - F_ref(x) |
# KL: categorical, asymmetric, needs Q(i)>0
# D_KL(P||Q) = sum P(i)*ln(P(i)/Q(i))
# -> prefer symmetric Jensen-Shannon (~0.1)
# MMD: high-dim embeddings, no binning (RBF kernel)
# MMD^2 = E[k(x,x')] + E[k(y,y')] - 2E[k(x,y)]
# default threshold ~0.015 (Evidently)
```

Data Contracts I — chuẩn ODCS

Hợp đồng = lời hứa (YAML trong git). Test = thực thi.

Hành trình: template YAML nội bộ **PayPal** → open-source 2023 → **Bitol** (LF AI & Data, 11/2023) → v3.0 (10/2024) → **v3.1.0 (12/2025)**.



~11 khối top-level (máy đọc được, versioned như code)

fundamentals · schema · quality · slaProperties · support · team · roles · servers · price · tags · customProperties. Mới ở 3.1.0: **Relationships** (FK), JSON Schema chặt hơn, **Scheduled SLAs** (thực thi, không chỉ tài liệu) — 0 breaking change vs 3.0.


```
apiVersion: v3.1.0
kind: DataContract
version: 1.2.0
schema:
  # <- schema-stable
  - name: orders
    # physical: analytics.daily_orders
    properties:
      - {name: order_id, logicalType: string, required: true, unique: true}
      - {name: amount, logicalType: number, required: true}
  quality:
    # <- distribution / complete
    - name: amount_non_negative
      type: sql
      query: SELECT COUNT(*) FROM analytics.daily_orders WHERE amount < 0
      mustBe: 0
  slaProperties:
    # <- freshness
    - {property: latency, value: 24, unit: h}
  team: [{username: data-eng, role: owner}]
  support: [{channel: "#data-orders", tool: slack}]
```

Ảnh xạ

schema → schema-stable;
quality → complete/distribution;
slaProperties.latency → freshness; team/support → owner để gọi khi vỡ.

Chạy (Data Contract CLI)

datacontract lint → test (kết nối nguồn thật) → changelog v1 v2 (phát hiện breaking change). v1.x giữa-2026, native ODCS, 28 định dạng export.

Data Contracts II — thực thi & shift-left

Catalog có YAML $\neq$ thực thi. 3 tầng enforcement.

08

dbt model contracts (batch)

GA từ dbt 1.5. `contract.enforced: true + columns (name/data_type/constraints)`. dbt sinh DDL; **preflight** fail build nếu cột/type lệch. CI bắt breaking change qua `state:modified.contract`.

Postgres thực thi tất cả; Snowflake/BQ/Databricks thực thi `not_null` lúc build.

Confluent “Data Contracts” (v7.4+): rule CEL chạy lúc WRITE/READ (vd `size(message.ssn)==9`); `migrationRules` JSONata; fail → DLQ.

Schema Registry (streaming)

Một đăng ký không hợp lệ bị **từ chối**:

- **BACKWARD** (mặc định): consumer mới đọc data cũ → update consumer trước
- **FORWARD**: consumer cũ đọc data mới
- **FULL**: cả hai; `_TRANSITIVE` = vs mọi version

BACKWARD ưu tiên cho Kafka (replay từ đầu topic).

Lưu ý: “Catalog không có ý kiến gì về data chảy qua nó. Nếu câu trả lời là ‘catalog đã có file YAML’ thì công việc còn CHƯA bắt đầu.” — một góc nhìn của giới hành nghề (Zircote, 2026), không phải số liệu đo lường.

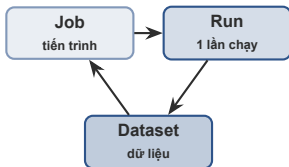
Tầng	Cơ chế	Công cụ
Shift-left CI gate	chặn TRƯỚC khi deploy	datacontract-cli, Buf (buf breaking)
Streaming runtime	xác thực schema-ID phía broker	Redpanda (OSS, broker-side) vs Confluent (Enterprise)
Batch post-ingest	scheduled validation	OpenMetadata 1.10+ (OSS) / DataHub Cloud

Cảnh báo tinh tế

Confluent SR *open-source* kiểm tra **phía client** — một producer cũ/raw có thể *vượt qua*. Thực thi phía broker thật mới chặn được.

Lineage I — OpenLineage & mô hình sự kiện

Lineage là SỰ KIỆN, không phải bản quét hằng đêm. Chất lượng cuối cùng dòng sự kiện.



LF AI & Data — Graduate project

Contributed 5/2021 (Datakin), graduate 7/2023. Lỗi = JSON event spec (chỉ định nghĩa **wire format**, tách khỏi storage/UI).

3 loại event: RunEvent (START + COMPLETE/FAIL/ABORT) · DatasetEvent (đổi schema ngoài run) · JobEvent. Run ID = UUIDv7. Client openlineage-python ~1.50 (6/2026).

Bắt tự động ở tầng engine (không sửa code DAG)

Airflow (provider từ 2.7) · Spark (SparkListener đọc query plan) · Flink (agent streaming) · dbt (dbt-ol mỗi model + ref()/source()).

```
{ "eventType": "COMPLETE",
  "job": {"namespace":"etl.prod","name":"orders.daily_aggregate"},
  "outputs": [{
    "namespace":"snowflake://acct","name":"analytics.daily_orders",
    "facets": {
      "columnLineage": {                // <- which inputs feed
        each col
        "fields": { "total_amount": {
          "inputFields": [
            {"name":"raw.orders","field":"amount"} ] } } },
      "schema": { "fields": [
        {"name":"order_date","type":"DATE"},
        {"name":"total_amount","type":"DECIMAL"} ] } },
      "dataQualityMetrics": {           // <- freshness/volume/
        quality
        "rowCount": 15234,
        "columnMetrics": {"amount": {"nullCount":0,"distinctCount"
          :8123}} }
    } ] }
```

Điểm mấu chốt

Freshness/volume/quality là **facets** GẮN trên lineage event. → Lineage & quality là **MỘT** dòng sự kiện, không phải 2 hệ thống tách rời.

Marquez (lab target)

Server tham chiếu OpenLineage + UI.
./docker/up.sh → UI localhost:3000.
macOS: dùng --api-port 9000 để né cổng AirPlay.

Lineage II — catalog, column-level, impact

Catalog hiện đại nói OpenLineage. Lineage = root-cause + blast-radius.

10

Xuôi = Impact / “blast radius”

“Nếu cột này hỏng/đổi, **dashboard/model/report** nào vỡ?” → đánh giá rủi ro trước khi deploy.

Ngược = Root cause

“Bảng này sai — cái gì **nuôi** nó?” → tìm nguồn lỗi trong phút thay vì giờ.

- TTR thực tế **~9 giờ** (Monte Carlo 2022, n=300+); khảo sát sau cho ~15h — 9h là số thận trọng.
- “Nhanh hơn 58%” khi resolve outage = IDC, **DataHub tài trợ, n=5** — trình bày như số do vendor đặt hàng, mẫu nhỏ.

Lưu ý: Đừng nói quá “biến sự cố nhiều giờ thành vài phút”. Bằng chứng tốt nhất là *nhanh hơn ~58%* ($\approx 9h \rightarrow \approx 3,8h$). 3 cơ chế bắt lineage đều bỏ sót: PATH-referenced tables, UDF mờ, transform non-SQL → **tự động 90%, chú thích tay 10%**.

Nền tảng	2025–26 có gì mới
DataHub	1.0 (1/2025); Acryl đổi tên → DataHub, Series B \$35M; push (OpenLineage) hoặc pull
OpenMetadata	GA 1.13.x; cascade parser SqlGlott → SqlFluff → SqlParse ; Lineage Agent
Unity Catalog	OSS (LF, 6/2024); auto column-level cho mọi query, gồm cả AI assets (model/feature)
Cloud-native	SageMaker/DataZone & Purview nói OpenLineage — hyperscaler cũng theo chuẩn
Atlas / Collibra	<i>ingest</i> OpenLineage chứ không thay thế — phía “buy” của build-vs-buy

Lưu ý: Unity Catalog: column-level lineage **KHÔNG** bắt được khi bảng tham chiếu bằng *PATH* (chỉ bắt khi tham chiếu bằng tên bảng); chỉ hiện data bắt sau 1/9/2024.

Pipeline: parse SQL → **AST** (đa số dùng sqlglot, 30+ dialect)
→ qualify tên bảng → lấy schema thật từ metadata → giải mã
cột mở hồ/SELECT * → phát sinh cạnh cột.

Schema-awareness là điểm khác biệt trên CTE, subquery,
SELECT *, join.

Benchmark của DataHub (column-level SELECT):

Parser	Độ chính xác
DataHub (schema-aware)	97%
sqllineage	42%
openlineage-sql	35%

*Khi không có SQL → dựng lại từ **query logs**.*

AI-Infra I — Feature Stores

Training-serving skew, freshness ceiling, point-in-time correctness.



Skew = biểu diễn feature lúc train khác lúc serve. Feature store *giảm* chứ **không xoá** skew (chúng hợp nhất *định nghĩa*, không hợp nhất *thực thi*: đường batch offline vs đường low-latency online phân kỳ).

Nước đi quan sát: log feature *thực sự* được serve rồi key-join ngược về batch train.

Databricks (2/2025)

Endpoint serving có thể log “augmented DataFrame” chứa feature values đã lookup → **inference table** = ground-truth feature đã serve để diff với training.

```
# 1) Skew: served features vs training batch
served = read_inference_table() # actual served
                                vectors
train  = read_training_batch()
for f in feature_cols:
    psi = population_stability_index(train[f], served
                                     [f], bins=10)
    assert psi < 0.25, f"skew on {f}: PSI={psi:.3f}"

# 2) Online/offline value consistency (point-in-time)
mismatch = 0
for k, t in sample_entity_keys(1000):
    on = online_store.get(k, "x")
    off = offline_store.get_asof(k, "x", asof=t) #
                                                AS OF
    if abs(on - off) > TOL: mismatch += 1
consistency = 1 - mismatch/1000 # SLO e.g. >=
                                0.999
```

Trần freshness

Online store chỉ serve giá trị **đã materialize** → lịch materialization *chặn trên* độ tươi. staleness = now() - feature_event_time so với SLA. CDC (Debezium) là đồn bầy 2025 cho sub-minute.

Point-in-time correctness

Hàng training phải join feature **as-of** nhãn (feature_ts <= label_ts) — AS OF / time-travel join. Thuật ngữ đúng: **temporal/look-ahead leakage**.

```
# 3) Point-in-time leakage guard (anti-join)
# No feature timestamp may exceed its label timestamp.
assert (features.feature_event_time
        <= labels.label_event_time).all(), \
        "look-ahead leakage detected"

# All 5 stores expose AS-OF joins:
#   Feast get_historical_features
#   Databricks time-series FeatureLookup
#   Tecton / Hopsworks (Hudi) / SageMaker FS
```

Lưu ý: Số kiểu “offline AUC 0,95 → on-line 0,78” chỉ là **minh họa** từ 1 tutorial không nguồn — đừng trích như đo đạc. On-demand FeatureFunctions (tính lúc inference) là **mặt skew khó nhất**.

Store	Cơ chế DQ / drift (đã định chính)
Feast	GE-based DQM đã deprecated trong roadmap; transforms on-read/on-write Beta, streaming Alpha
Tecton	native GE DQM + freshness/materialization alerts; tích hợp Fiddler cho drift
Databricks	FeatureSpec trong Unity Catalog; Declarative Feature APIs (2025); inference-table logging
Featureform	infra-agnostic Virtual Feature Store (KHÔNG “trên Iceberg”); gia nhập Redis (10/2025)
Hopsworks	time-travel qua Hudi (<code>point_in_time_join</code>)

Hệ quả quan sát

Online/offline consistency (công thức: xem code 2 slide trước) là một SLO *riêng*, tách khỏi freshness. Phân kỳ dai dẳng = bug materialization hoặc 2 đường code (online/offline) đã lệch nhau — không tự lành, cần alert riêng.

AI-Infra II — Vector/Embedding & RAG

Embedding drift, corpus freshness “silent failure”, re-embedding.

Phương pháp	Ngưỡng / khi dùng
Euclidean centroid	một-số nhanh
Cosine centroid	scale-invariant
Domain classifier (AUC)	alert ~ 0,55 — <i>mặc định</i>
Share of drifted dims	Wasserstein/dim
MMD (§6)	0,015; high-dim, chậm nhất

Arize Phoenix: Euclidean centroid + UMAP 3D + HDBSCAN để khoanh cụm nào drift.

```
# Domain classifier = recommended default
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
import numpy as np
X = np.vstack([ref_emb, cur_emb])
y = np.r_[np.zeros(len(ref_emb)), np.ones(len(cur_emb))]
auc = cross_val_score(LogisticRegression(max_iter=1000),
                      X, y, cv=5, scoring="roc_auc").mean()
# ~0.5 no drift | ~0.55 alert | ->1.0 strong drift

# one-number centroid distance (Phoenix/Evidently)
euclid = np.linalg.norm(ref_emb.mean(0) - cur_emb.mean(0))
```

Lưu ý: Cosine similarity tính thuần trong không gian ngữ nghĩa — **không có chiều thời gian**. Một chunk 18 tháng tuổi vẫn “điểm cao” như chunk mới. Model lấy ra tài liệu cũ mà mọi metric vẫn xanh (arXiv 2509.19376, 2025).

4 monitor chạy được (ngưỡng là mặc định để tinh chỉnh, KHÔNG phải hằng số):

- ingestion_timestamp + last_verified TTL theo lớp nội dung: ~**6 tháng** cho doc kiến trúc/nền tảng; ~2–4 tuần cho API/release notes.
- Weekly top-k nearest-neighbor overlap (85–95% khoẻ / <70% mất mát — minh hoạ).
- Orphan-chunk GC + document-TTL.
- Cosine-distance band trên ref đã re-embed (minh hoạ).

Re-embed sau khi đổi model

Cosine **KHÔNG so sánh được** giữa các thể hệ embedding model — trộn 2 generation trong 1 index → retrieval suy giảm âm thầm. Patterns: shadow deploy · lazy/parallel re-embed · CDC selective.

Drift-Adapter (EMNLP 2025): map query mới về không gian cũ, hồi 95–99% recall, $<10\mu s$, $\sim 100\times$ tiết kiệm tính lại.

1 metric sức khỏe / vector DB

- **Qdrant**: /telemetry + Prometheus /metrics
- **Weaviate**: vectorQueueLength (backlog chưa index)
- **pgvector**: không có recall sẵn → so HNSW xấp xỉ vs exact; tính chính hnsw.ef_search

Lưu ý: Versioning bắt buộc: gắn mỗi vector embedding_model_id + chunker config + source-doc id → truy được “blast radius” khi upgrade. Đừng bao giờ so cosine khác version.

Reference-free (chạy trên live traffic)

Ragas: Context Precision / Recall / Entities Recall / Noise Sensitivity — không cần gold answer. **Phoenix:** gán DocumentEvaluations vào retriever span (LLM judge từng chunk relevant?).

Khi có nhãn

IR cổ điển: **Hit Rate, MRR, NDCG@k** (đã học ở Ngày 14/19). Bỏ trợ reference-free khi không có ground truth.

Profile text, không phải vector

WhyLabs **langkit** trích tín hiệu prompt/response (sentiment, toxicity, PII, jailbreak-similarity, relevance) → **whylogs** ghi profile nhẹ theo bucket → drift monitor, *không lưu raw text*.

Vì sao cần

Bắt được retriever *âm thầm* ngừng trả đúng chunk — ngay cả khi generation “nhìn vẫn ổn”.

AI-Infra III — Training / LLM / Agent data

Contamination, dedup, label errors, LLM-as-judge, provenance.



Lưu ý: Câu “MMLU $\sim 29\%$ nhiễm, rót ~ 13 điểm trên tập sạch” là **ghép 2 nghiên cứu, 2 benchmark khác nhau** bởi một blog. Dùng đúng:

- **MMLU 29,1%** = Deng et al. (arXiv 2311.09783, NAACL'24) qua probe **TS-Guessing** — là tín hiệu *ghi nhớ*, KHÔNG phải tỷ lệ trùng lặp verbatim hay mức rót.
- **Rót ~ 13 điểm** = **GSM1k** (arXiv 2405.00332, Scale AI) trên bản sao mới của **GSM8K** — với họ model overfit nhất (Phi/Mistral), KHÔNG phải MMLU.

Công cụ & benchmark chống nhiễm

LiveBench (ICLR'25, refresh hằng tháng), DyCodeEval; lm-eval-harness có decontamination n-gram (chạy được trên lab). *Đừng trích arXiv 2605.19999 (future-dated).*

Dedup + quality filtering

(GPU/Ray-native, 2025–26)

- **DataTrove** (HF): MinhashDedup (LSH), nuôi FineWeb
- **NeMo Curator**: dedup + quality classifier; tái kiến trúc trên **Ray** (26.02, 2/2026)
- FineWeb-2 (12/2024): per-language scorer

Label-error detection

Cleanlab (confident learning): điểm chất lượng nhãn 0–1 mỗi mẫu, model-agnostic (text/image/tabular). Tìm lỗi hệ thống trong ImageNet/MNIST.

Argilla (HF): curation no-code cho SFT/preference data.

Lab: chạy cleanlab, sort theo điểm, soi 50 mẫu tệ nhất.

Lưu ý: “Reward signal cũng là DATA”: nhiều annotation RLHF thường $>20\%$; nhất quán human-vs-expert $\sim 70\%$. Lọc + golden questions để chấm annotator.

LLM-as-judge áp cho DATA QUALITY

Nhiều LLM chấm mỗi data point; giữ điểm consensus-valid — bắt được *validation ngữ nghĩa* rule không biểu đạt nổi (vd “ticket này có thật sự về billing?”).

Lưu ý: Bias đã ghi nhận: verbosity, position, self-enhancement → LLM-judge ở quy mô + **human review** có chủ đích, neo vào ground-truth nhỏ (100–200 mẫu).

Hallucination = bài toán provenance

“Correctness $\neq$ Faithfulness” (arXiv 2412.18004).
→ **Truy hallucination ngược về corpus** qua lineage.

Agentic data observability (frontier 2026)

Quan sát **data agent ĐỌC & GHI** — tool outputs, memory, scratchpad, retrieved context — qua *decision provenance*. Tách fact người-nói khỏi suy luận agent-sinh. Croissant + **MCP** (10/2025): LLM tìm/nạp dataset qua MCP.

Góc Ngày-27 = nội dung & xuất xứ của data, KHÔNG phải telemetry runtime của agent (đó là Ngày 23).

Frontier & chi phí

Đọc metadata thay vì scan; streaming; trust score; FinOps; agentic DQ.

14

- **Iceberg**: snapshot summary mang total/added-records, added-files/commit (volume+freshness *miễn phí*); manifest có min/max/null-count/record-count theo cột; \$snapshots/\$partitions/\$history truy vấn SQL được.
- **Delta**: tương đương trong transaction log (numRecords, minValues, nullCount) + \$history.
- **Iceberg Puffin**: stats file cho **distinct-count (NDV)** xấp xỉ theo cột *không scan* → monitor cardinality/uniqueness trên bảng triệu file.

Databricks Lakehouse Monitoring

3 profile khai báo: **Snapshot / TimeSeries / InferenceLog**. Một create_monitor → tự sinh bảng profile + drift metric + dashboard (data drift, prediction drift, null%, distinct-count, schema change).

Gotcha: profile TimeSeries/Inference chỉ tính trên **30 ngày** gần nhất.

Vì sao quan trọng

Đọc catalog/manifest tốn **gần-0 compute** — nền tảng cho FinOps observability (slide kế).

Streaming = kỷ luật riêng

- **Watermark progress** + tỷ lệ event trễ/lệch thứ tự (Flink): late tăng = completeness giảm âm thầm.
- **Kafka consumer lag** = proxy freshness.
- **Schema Registry reject rate** = contract tại ingest.

Batch freshness = last load time vs stream freshness
= watermark/lag.

Trust score + circuit breaker

Điểm sức khỏe 0–100 (trọng số các chiều) gắn vào catalog → consumer thấy độ tin cậy TRƯỚC khi query. **Circuit breaker**: dừng pipeline khi score < ngưỡng (vd <70).

Lưu ý: Một con số **che mắt** chiều nào hổng — luôn cần drill-down.

4 đòn bẩy chi phí

1. Đẩy check sang **metadata** (snapshot/manifest/Puffin) thay vì COUNT(DISTINCT) full-scan — warehouse compute mới là hoá đơn thật.
2. **Sample** bảng lớn cho test phân phối (PSI/Wasserstein chịu được sampling).
3. **Giới hạn cardinality** metric (user_id/txn_id làm nỏ time-series).
4. **Phân tầng** tần suất monitor theo giá trị asset (gold daily, raw hourly).

Agentic DQ (headline 2026)

Databricks **Data Quality Monitoring** (4/2/2026, public preview): monitor schema-level 1-click; agent học mẫu lịch sử+mùa vụ; dùng **Unity Catalog lineage** + query volume để ưu tiên bảng high-impact. Monte Carlo monitoring/triage agents; Acceldata DQ Agent.

Cung bậc Maturity

Các ví dụ trên = bước nhảy **Level 2** → **Level 4** trên thang trưởng thành (xem §2).

Tuân thủ, sự cố & Lab 27

Lineage = “siêu năng lực” cho xoá dữ liệu & xác định phạm vi rò rỉ.

15

Việt Nam — PDPL (Luật 91/2025/QH15)

Ban hành 26/6/2025, **hiệu lực 1/1/2026** (Nghị định 356/2025). **DPIA** trong 60 ngày kể từ khi xử lý; **CTIA** trong 60 ngày từ lần chuyển ra nước ngoài đầu tiên; **kiểm toán A05** hằng năm; phạt tới **5% doanh thu năm trước** (cross-border).

EU AI Act — Art. 53(1)(d)

Template tóm tắt nội dung huấn luyện (24/7/2025);
bộc khai nguồn data → **training-data lineage** thành
yêu cầu pháp lý.

Vì sao lineage là “siêu năng lực”

Một graph lineage trả lời CẢ HAI:

- **Xoá (PDPL/GDPR Art.17):** “phải xoá mọi bản sao/dẫn xuất Ở ĐÂU?”
- **Phạm vi rò rỉ:** “truy xuôi tới mọi bảng/report/model bị ảnh hưởng”.

Không có lineage → xoá sót → *bản thân việc đó là vi phạm*.

Xác nhận lại số nghị định/ngày trước khi giảng (cờ needs-verify).

LIVE DEMO

1. **Demo 1:** Inject schema change vào upstream → GX 1.0 checkpoint fail → chặn pipeline + Slack alert.
2. **Demo 2:** Inject volume anomaly (10% bình thường) → MAD/decompose-then-detect bắt → alert (không báo giả cuối tuần).
3. **Demo 3:** dbt/SQL transform → phát OpenLineage event → Marquez UI → “cột `total_amount` hỏng thì report nào vỡ?” (impact).
4. **Demo 4:** Đổi embedding model → domain-classifier AUC 0,5 → 0,8 → cảnh báo embedding drift trên corpus RAG.
5. **Flow:** alert → lineage root-cause → contract/check chặn → verify.

- ✓ Mỗi nguồn quan trọng có check 5 trụ cột (freshness/volume/distribution/schema/lineage).
- ✓ Ít nhất 1 **data contract** (ODCS) cho interface producer→consumer trọng yếu, test trong CI.
- ✓ Anomaly detection **decompose-then-detect** (không ngưỡng tĩnh) cho metric có mùa vụ.
- ✓ **OpenLineage** bật ở engine (Airflow/Spark/dbt) → impact + root-cause query được.
- ✓ Feature store: monitor **training-serving skew** + online/offline consistency SLO.
- ✓ RAG: **embedding drift** (AUC) + corpus freshness TTL + version theo embedding_model_id.
- ✓ FinOps: đẩy check sang metadata, sample bảng lớn, phân tầng tần suất theo giá trị asset.
- ✓ Lineage phục vụ tuân thủ: trả lời được câu hỏi *xoá & phạm vi rò rỉ* (PDPL).

LAB #27

Mục tiêu: Xây Data Observability + Lineage chạy được (zero-key, DuckDB)

Deliverable:

- **Track 1 — Checks:** GX 1.0 / Soda 4.0 checkpoint chặn pipeline khi vi phạm freshness/volume/distribution/schema.
- **Track 2 — Contract:** `orders.odcs.yaml` + `datacontract lint/test/changelog` trong CI.
- **Track 3 — Lineage:** OpenLineage events → Marquez (`docker/up.sh`; macOS `--api-port 9000`); impact & root-cause.
- **Track 4 — AI-infra:** embedding-drift (AUC) + training-serving skew (PSI).
- **Bonus:** decompose-then-detect (Prophet/STL) + trust score 0–100.

Thời gian: 2.5h

Những ý chính cần nhớ trước khi sang bài tiếp theo

1

Data observability $\neq$ system monitoring. “Pipeline thành công” + “dashboard xanh” không chứng minh data đúng — **schema checks chỉ bắt 7,8% sự cố**.

2

Contract = lời hứa (YAML/git); **test** = thực thi; **lineage** = sự kiện, chất lượng cuối cùng dòng OpenLineage. Lineage cho impact + root-cause + tuân thủ.

3

AI-infra cần chiều quan sát riêng: training-serving skew, embedding/RAG drift, contamination, label quality — và **rules** $\rightarrow$ **ML** $\rightarrow$ **agents**.

4

Luôn truy số liệu về nguồn (nhiều “stat kinh điển” là folklore) và đẩy check sang **metadata** để rẻ (FinOps).

Bài tiếp theo

Ngày 28: Platform Engineering & Documentation — Milestone 3

“Ghép mọi mảnh N16–N27 thành một platform; runbooks, golden paths, C4 diagrams; demo end-to-end.”

Bài tập về nhà

- Hoàn thành Lab 27 (4 tracks): checks + contract + lineage + AI-infra drift.
- Chuẩn bị data-observability & lineage làm một mục trong tài liệu platform.
- Rà soát toàn bộ stack cho Milestone 3 demo.

- **OpenLineage** spec/object-model · **Marquez** quickstart · OpenLineage CHANGELOG (v1.50, 6/2026).
- **ODCS v3.1.0** (bitol.io) · **Data Contract CLI** (datacontract.com) · Confluent schema evolution.
- **Great Expectations** GX Core 1.0 blog · **Soda Core 4.0** release notes · Pandera · Elementary.
- **Evidently** embedding/data drift · **Arize Phoenix** · Ragas · WhyLabs langkit.
- Feast / Tecton / Databricks Feature Store · Databricks Lakehouse Monitoring & DQM (2/2026).
- Monte Carlo: 5 pillars, 2026 Data Quality Statistics · IBM IBV cost-of-poor-data 2025.
- Incidents: Unity 8-K (5/2022), Equifax NY AG (1/2025), Zillow (11/2021), Citigroup (2/2025).
- Tuân thủ: VN **PDPL Luật 91/2025** · EU AI Act Art.53(1)(d).

Chi tiết nguồn + cảnh báo folklore: RESEARCH-day27-data-observability-lineage.md.

Hỏi & Đáp

Câu hỏi về data observability, contracts (ODCS), lineage (OpenLineage), hay quan sát feature/vector/training data?



Cảm ơn!

AICB-P2T2 · Ngày 27

Data Observability & Lineage

lms.vinuni.edu.vn · Slide & Lab 27 trên LMS